Exercício 8.4: Desafio: processamento de fila com escala a zero (KEDA)

Aluno: Luiz Brito · **Data:** 24/10/2026 · **Disciplina:** Cloud Native Development (CP3)

Ambiente

- Cluster kind com 3 nós, Kubernetes v1.36.1, namespace `store-dev`.
- **Trilha A (KEDA).** KEDA 2.21.0, instalado com o comando do enunciado (`helm install keda kedacore/keda -n keda --create-namespace`).
- Redis `redis:7-alpine` como fila (lista `pedidos`). Sequência em `04-commands.sh`, com amostras a cada 5 s.

Manifestos

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
  namespace: store-dev
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
        - name: redis
          image: redis:7-alpine
          ports:
            - containerPort: 6379
---
apiVersion: v1
kind: Service
metadata:
  name: redis
  namespace: store-dev
spec:
  selector:
    app: redis
  ports:
    - port: 6379
```

```yaml
apiVersion: keda.sh/v1alpha1
kind: ScaledJob
metadata:
  name: store-worker-fila
  namespace: store-dev
spec:
  pollingInterval: 10
  minReplicaCount: 0
  maxReplicaCount: 10
  successfulJobsHistoryLimit: 0
  failedJobsHistoryLimit: 2
  jobTargetRef:
    backoffLimit: 2
    template:
      metadata:
        labels:
          app: store-worker
      spec:
        restartPolicy: Never
        containers:
          - name: worker
            image: redis:7-alpine
            # cada Job consome ate 5 pedidos (= listLength) e termina
            command: ["sh", "-c"]
            args:
              - |
                for i in 1 2 3 4 5; do
                  p=$(redis-cli -h redis LPOP pedidos)
                  [ -z "$p" ] && break
                  echo "$(hostname) processou $p"
                  sleep 6
                done
  triggers:
    - type: redis
      metadata:
        address: redis.store-dev.svc.cluster.local:6379
        listName: pedidos
        listLength: "5"
```

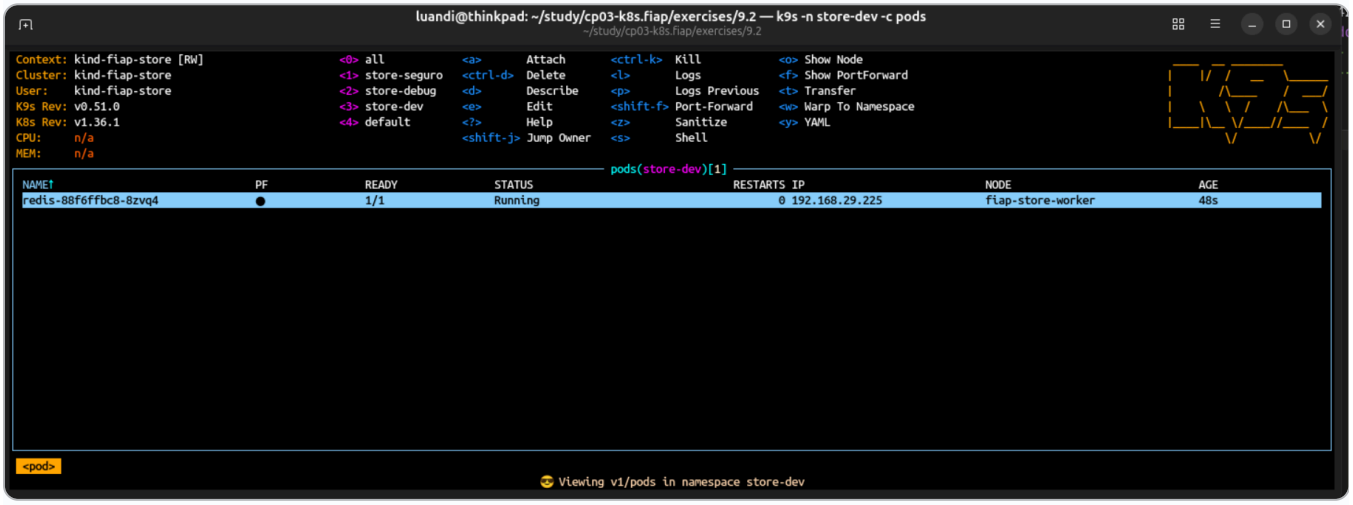
Decisões:

- **ScaledJob, não ScaledObject.** Cada lote é um Job que termina. Com Deployment + ScaledObject, o KEDA reduziria réplicas no meio do processamento e mataria Pods com itens em andamento.
- **Cada Job consome até 5 itens (= `listLength`) e sai.** Com a estratégia padrão, o KEDA cria `ceil(fila / listLength)` Jobs, então o trabalho de cada Job casa com a conta do scaler: 50 itens / 5 = 10 Jobs.
- `successfulJobsHistoryLimit: 0`. Jobs concluídos são removidos, e é isso que faz `get pods` voltar a vazio, e não a uma lista de `Completed`.

- `sleep 6` **por item** deixa o pico longo o bastante para ser observado e fotografado.

(a) Fila vazia: zero Pods

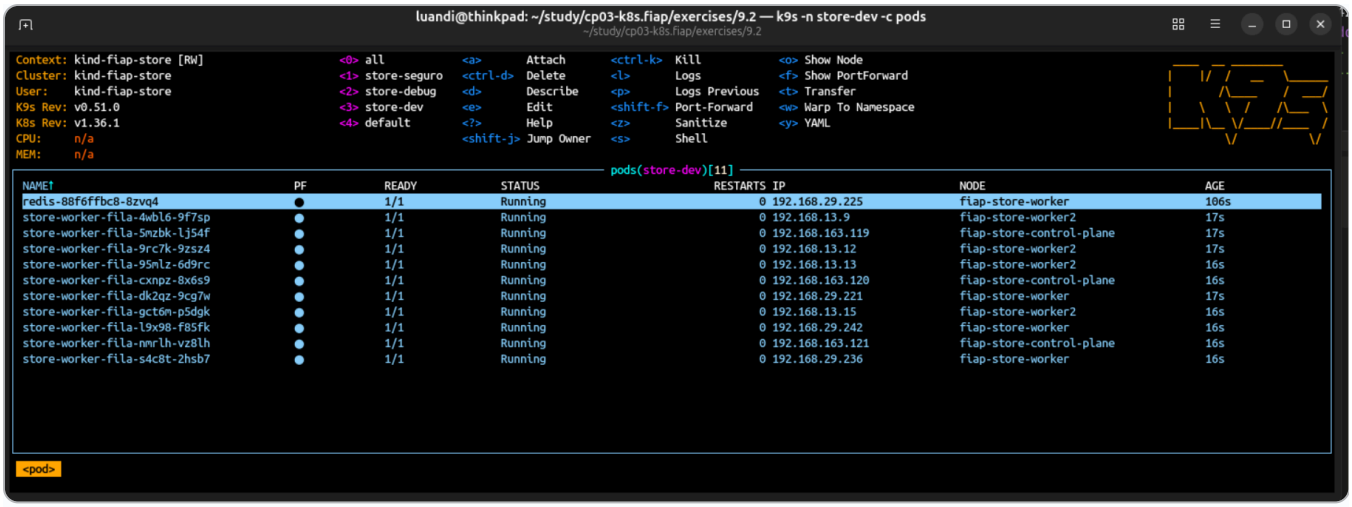
```
### 01:34:07 fila=0
$ kubectl get pods -n store-dev -l app=store-worker
No resources found in store-dev namespace.
```



(b) 50 itens: o KEDA cria 10 Jobs

```
$ kubectl exec deploy/redis -- sh -c 'for i in $(seq 1 50); do redis-cli RPUSH pedidos "pedido-$i"; done'

### 01:34:08 fila=50     (0 Pods)
### 01:34:13 fila=50     (0 Pods)
### 01:34:18 fila=50
NAME                              READY   STATUS              RESTARTS   AGE
store-worker-fila-4wbl6-9f7sp     0/1     ContainerCreating   0          1s
store-worker-fila-5mzbk-lj54f     0/1     ContainerCreating   0          1s
store-worker-fila-95mlz-6d9rc     0/1     ContainerCreating   0          0s
store-worker-fila-9rc7k-9zsz4     0/1     ContainerCreating   0          1s
store-worker-fila-cxnpz-8x6s9     0/1     ContainerCreating   0          0s
store-worker-fila-dk2qz-9cg7w     0/1     ContainerCreating   0          1s
store-worker-fila-gct6m-p5dgk     0/1     ContainerCreating   0          0s
store-worker-fila-l9x98-f85fk     0/1     ContainerCreating   0          0s
store-worker-fila-nmrlh-vz8lh     0/1     ContainerCreating   0          0s
store-worker-fila-s4c8t-2hsb7     0/1     ContainerCreating   0          0s
```

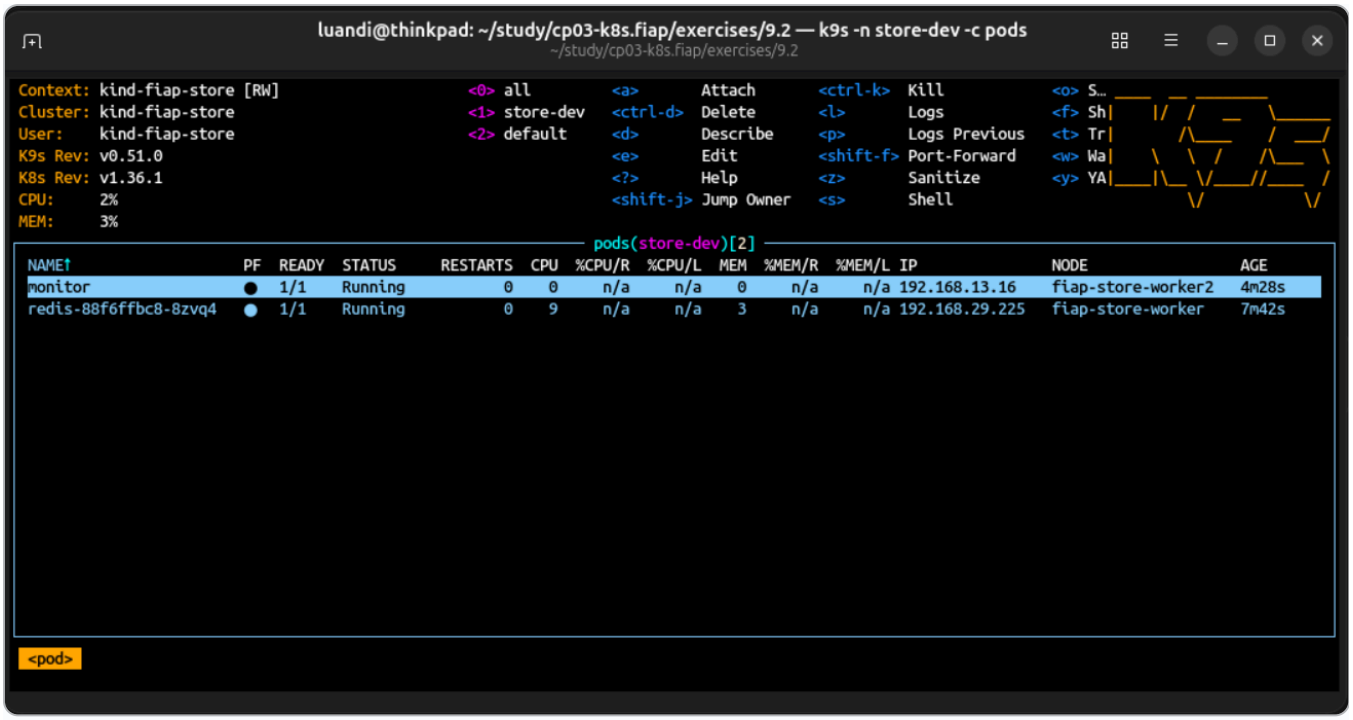


(c) Fila esvaziada: volta a zero

Pods ativos (`Running`, `ContainerCreating`, `Pending`) por amostra:

```
hora       fila   Pods ativos
01:34:07     0      0
01:34:08    50      0
01:34:13    50      0
01:34:18    50     10
01:34:24    47     10
01:34:29    37     10
01:34:35    27     10
01:34:40    18     10
01:34:45     8     10
01:34:51     3      7
01:34:56     0      5
01:35:01     0      0
```

```
### 01:35:24
$ kubectl get pods -n store-dev -l app=store-worker
No resources found in store-dev namespace.
$ kubectl get jobs -n store-dev
No resources found in store-dev namespace.
$ kubectl exec deploy/redis -- redis-cli LLEN pedidos
0
```



Ciclo 0 → 10 → 0 em 53 s. Foram 9 s de reação (01:34:08 → 01:34:17, dentro do `pollingInterval` de 10 s), 30 s de processamento (50 itens × 6 s / 10 workers) e a limpeza dos Jobs concluídos.

`describe scaledjob`

```
$ kubectl -n store-dev describe scaledjob store-worker-fila
Spec:
  Max Replica Count:     10
  Min Replica Count:     0
  Polling Interval:      10
  Successful Jobs History Limit:  0
  Triggers:
    Metadata:
      Address:      redis.store-dev.svc.cluster.local:6379
      List Length:  5
      List Name:    pedidos
    Type:           redis
Status:
  Conditions:
    Message:  ScaledJob is defined correctly and is ready for scaling
    Reason:   ScaledJobReady
    Status:   True
    Type:     Ready
    Message:  Scaling is not performed because triggers are not active
    Reason:   ScalerNotActive
    Status:   False
    Type:     Active
  Last Active Time:  2026-10-24T04:34:47Z
Events:
  Normal  KEDAJobsCreated     43s   scale-handler  Created 10 jobs
  Normal  ScaledJobActive     43s   scale-handler  Scaling is performed because triggers are active
  Normal  ScaledJobInactive   4s    scale-handler  Scaling is not performed because triggers are not active
```

`Created 10 jobs` num único ciclo de polling: exatamente `ceil(50 / 5)`, limitado pelo `maxReplicaCount: 10`.

Ponto fraco encontrado

O worker usa `LPOP`, que **tira o item da fila antes** de processá-lo. Se um Pod morrer depois do `LPOP` e antes de terminar, o pedido some, e o `backoffLimit` não ajuda, porque a nova tentativa vai buscar o **próximo** item. Em produção isso exige `LMOVE` para uma lista de processamento com reconciliação, ou uma fila com confirmação explícita (SQS, RabbitMQ). Ver a seção de falhas do `03-COMPARACAO.md`.

Comparação: Job fixo x KEDA ScaledJob x SQS + Lambda

Problema: processar a fila de pedidos da FIAP Store, com volume baixo o ano todo e pico na Black Friday.

Premissas usadas nas contas (todas explícitas para poderem ser trocadas):

- Cada pedido leva **2 s** de processamento e o worker usa **512 MB**.
- Preços on-demand em us-east-1, consultados na AWS Price List API em 24/10/2026:

Item	Preço
EKS control plane	US$ 0,10 por cluster-hora
EC2 `t3.medium` (2 vCPU, 4 GiB, Linux)	US$ 0,0416 por hora
Lambda requisições	US$ 0,20 por milhão
Lambda duração (x86, faixa 1)	US$ 0,0000166667 por GB-segundo
SQS standard (faixa 1)	US$ 0,40 por milhão de requisições

Um mês tem 730 h. Então o EKS custa **US$ 73,00/mês** e cada `t3.medium` ligado direto custa **US$ 30,37/mês**.

---

1. Pico de 10.000 mensagens

O trabalho total é 10.000 × 2 s = **20.000 s de processamento**.

- **Job com `parallelism` fixo:** a vazão é o `parallelism`. Com `parallelism: 10`, são 20.000 / 10 = 2.000 s, **cerca de 33 min**. Subir o número exige mudar o manifesto e ter nós para isso. Fora do pico, ou o Job não existe (alguém tem que criá-lo) ou fica com Pods ociosos.
- **KEDA ScaledJob:** o KEDA lê o tamanho da fila a cada `pollingInterval` e cria `ceil(fila / listLength)` Jobs, até o `maxReplicaCount`. Com `maxReplicaCount: 10` o teto é o mesmo do Job fixo (33 min). A diferença é que ele vai a zero sozinho e sobe sozinho. Com `maxReplicaCount: 100` o trabalho cai para cerca de 200 s, **se houver nós**. Num cluster sem folga, os Pods ficam `Pending` até o Cluster Autoscaler ou o Karpenter criarem EC2 novas. No pico, o gargalo real é o provisionamento de nó, não o KEDA.
- **SQS + Lambda:** pela documentação do event source mapping, o Lambda aumenta a leitura em até **300 invocações concorrentes por minuto**, até o máximo padrão de **1.250**. No primeiro minuto a média fica perto de 150 concorrentes (cerca de 9.000 s de trabalho) e no segundo perto de 450. Então os 10.000 pedidos saem em **cerca de 2 min**, sem nenhum nó para provisionar. A cota da conta (1.000 execuções concorrentes por padrão) é o limite a observar se outras funções disputarem a mesma conta.

---

2. Latência de cold start

- **Job fixo:** não tem cold start de escala, porque não escala. Cada Pod paga agendamento + pull de imagem (se não estiver em cache) + início do container. No lab,

um Pod `busybox` com imagem em cache sai de `Pending` para `Running` em 1 a 2 s.

- **KEDA ScaledJob:** soma o `pollingInterval` (padrão 30 s, configurado com 10 s aqui) ao tempo de criar o Job e subir o Pod. Medido no lab (8.4): o `RPUSH` dos 50 itens terminou às 01:34:08 e os 10 Jobs foram criados às 01:34:17. Foram 9 s de reação, dentro do `pollingInterval` de 10 s, e os Pods ficaram `Running` cerca de 6 s depois, com imagem em cache. Total de cerca de 15 s do primeiro item na fila até o processamento. Se o pico exigir nó novo, entra o tempo de boot de uma EC2 e de ela entrar no cluster, que é de minutos.
- **SQS + Lambda:** segundo a AWS, o cold start vai de **menos de 100 ms a mais de 1 s** e acontece em **menos de 1% das invocações** em cargas de produção. É a menor latência das três para sair do zero.

---

3. Limite de tempo por item

- **Lambda:** timeout máximo de **900 s (15 min)** por invocação. Um item que demore mais precisa ser quebrado (Step Functions, por exemplo) ou sair do Lambda.
- **Job e ScaledJob: sem limite** por padrão. O limite é opcional, via `activeDeadlineSeconds`. Um relatório de fechamento da Black Friday que leve 40 min roda sem mudança de arquitetura.

---

4. Modelo de custo

Piso fixo (custo com fila vazia):

Arquitetura	Piso mensal
Job fixo ou KEDA no EKS, 2 × `t3.medium`	73,00 + 2 × 30,37 = **US$ 133,74**
SQS + Lambda	**US$ 0** (paga por uso)

O KEDA zera os **Pods**, não os **nós**. Sem um autoscaler de nós junto, a conta da EC2 continua a mesma. E o control plane de US$ 73/mês não zera nunca.

Custo variável por pico de 10.000 pedidos no Lambda:

- Requisições: 10.000 × 0,20 / 1.000.000 = US$ 0,002
- Duração: 10.000 × 2 s × 0,5 GB = 10.000 GB-s × 0,0000166667 = US$ 0,167
- SQS: cerca de 3 chamadas por mensagem (enviar, receber, apagar) = 30.000 × 0,40 / 1.000.000 = US$ 0,012
- **Total: cerca de US$ 0,18**

Black Friday com 1 milhão de pedidos num dia: 1.000.000 GB-s × 0,0000166667 = US$ 16,67, mais US$ 0,20 de requisições e US$ 1,20 de SQS: **cerca de US$ 18**.

No EKS, o mesmo dia custa o piso mais os nós extras do pico. São 1.000.000 × 2 s = 556 h de CPU. Em `t3.medium` (2 vCPU), dá cerca de 278 nós-hora, ou US$ 11,57, se o autoscaler ligar e desligar com precisão. É parecido com o Lambda **no dia do pico**. A diferença está nos outros 364 dias, quando o piso de US$ 133,74/mês continua sendo cobrado.

---

5. Tratamento de falhas

- **Job fixo:** `backoffLimit` conta falhas do Job inteiro. Ao estourar, o Job vai a `Failed` com `BackoffLimitExceeded` (reproduzido no 8.3) e os itens restantes não são processados. O `podFailurePolicy` refina isso: exit code de erro de negócio vira `FailJob` na hora, e `DisruptionTarget` (Pod expulso por drain ou preempção) vira `Ignore` e não gasta tentativa. Não existe DLQ nativa: o item que falhou só aparece nos logs.
- **KEDA ScaledJob:** cada Job tem o próprio `backoffLimit` e o `podFailurePolicy` funciona igual. O ponto fraco está na fila, não no KEDA: com `LPOP` em lista Redis, o item sai da fila antes de ser processado. Se o Pod morrer no meio, o pedido some. Para ter garantia, é preciso `LMOVE` para uma lista de processamento com reconciliação, ou trocar o trigger para SQS, que o KEDA também suporta.
- **SQS + Lambda:** a mensagem só é apagada se a função terminar com sucesso. Se falhar, ela volta a ficar visível depois do `VisibilityTimeout`. Depois de `maxReceiveCount` tentativas, vai para a **DLQ** e fica lá para análise e reprocessamento (redrive), sem perda. Com `ReportBatchItemFailures`, só os itens com falha do lote voltam para a fila.

Recomendação para a FIAP Store

SQS + Lambda para o processamento de pedidos, mantendo a `store-api` no Kubernetes.

1. O perfil de carga é exatamente o caso do Lambda: ocioso quase o ano todo e com um pico curto e intenso. Fora do pico, o custo é praticamente zero. No pico, a escala chega a mais de 1.000 concorrentes em poucos minutos, sem depender de provisionar nós.
2. Processar um pedido leva segundos, muito abaixo do limite de 15 min.
3. SQS com DLQ dá garantia de entrega e reprocessamento sem código extra. Com Redis + `LPOP`, isso teria de ser construído à mão.

O KEDA ScaledJob fica como alternativa para o que **não** cabe no Lambda: tarefas de mais de 15 min (relatórios, reconciliação) que podem rodar no cluster que já existe para a `store-api`, consumindo a mesma SQS pelo trigger `aws-sqs-queue`. O Job com `parallelism` fixo só se justifica para lotes previsíveis e agendados (via CronJob), nunca para fila com pico.

Fontes

- AWS Price List API, `AWSLambda`, `AmazonEKS`, `AWSQueueService` e EC2 on-demand, us-east-1, consultadas em 24/10/2026
- https://docs.aws.amazon.com/lambda/latest/dg/services-sqs-scaling.html
- https://docs.aws.amazon.com/lambda/latest/dg/gettingstarted-limits.html
- https://docs.aws.amazon.com/lambda/latest/dg/lambda-runtime-environment.html
- https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-dead-letter-queues.html
- https://kubernetes.io/docs/concepts/workloads/controllers/job/
- https://keda.sh/docs/2.21/reference/scaledjob-spec/
- https://keda.sh/docs/2.21/scalers/redis-lists/
- https://keda.sh/docs/2.21/scalers/aws-sqs/
- https://docs.aws.amazon.com/lambda/latest/dg/services-sqs-errorhandling.html

Critérios de aceite

Critério	Resultado
Três capturas de `get pods` provando zero → N → zero	ok (texto e 3 prints)
`COMPARACAO.md` com os cinco eixos, números concretos e recomendação	ok (preços da AWS Price List API, mais de 500 palavras)